

Text Vectorization & Word Embeddings

BoW · TF-IDF · Word2Vec · GloVe · FastText · Keras Embeddings

Rahul Basu

NLP · PES University

Session 2

Rahul Basu

Recap &
Motivation

BoW &
TF-IDF as
Baselines

Distributional
Hypothesis

Word2Vec

GloVe

FastText

Document-
Level
Embed-
dings

Keras
Embedding
Layer

Comparison
& Decision
Guide

Session
Summary

- 1 Recap & Motivation
- 2 BoW & TF-IDF as Baselines
- 3 Distributional Hypothesis
- 4 Word2Vec
- 5 GloVe

- 6 FastText
- 7 Document-Level Embeddings
- 8 Keras Embedding Layer
- 9 Comparison & Decision Guide
- 10 Session Summary

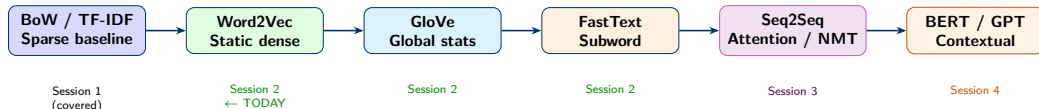
Session 1 Recap — What We Already Know

Concept	Key idea	Limitation
Tokenization	Split text into atomic units	Tokenizer choice matters
Stemming/Lemma	Reduce inflections to base form	Over-stemming merges meaning
Stopwords	Remove high-freq function words	Domain-specific lists differ
Bag of Words	Count vector, $\text{dim} = V $	Orthogonal, sparse
N-grams	Contiguous sequences as features	Vocabulary explosion
TF-IDF	Weight = $\text{TF} \times \log \frac{N+1}{n_t+1} + 1$	Still sparse, no semantics

The Fundamental Gap

In a 50,000-word vocabulary, “**king**” and “**queen**” are *exactly* as far apart as “**king**” and “**carrot**”. Both pairs have cosine similarity = 0. TF-IDF fixes stopwords noise but cannot fix geometric orthogonality.

The Road Map for Session 2



Sparse representations (Session 1)

- High-dimensional ($|V| \approx 50\text{K} - 1\text{M}$)
- Interpretable: each dim = a term
- Strong baselines on short/medium docs
- No semantic proximity

Dense embeddings (this session)

- Low-dimensional (50–300)
- Semantic proximity baked in
- Analogy arithmetic:
 $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$
- Pre-trained: transfer from huge corpora

```
from sklearn.feature_extraction.text \
    import TfidfVectorizer
from sklearn.linear_model import \
    LogisticRegression
from sklearn.model_selection import \
    cross_val_score

tv = TfidfVectorizer(
    min_df=5, max_df=0.9,
    max_features=8000,
    sublinear_tf=True,      # 1+log(f)
    ngram_range=(1,2),     #
                           uni+bigrams
)
X = tv.fit_transform(corpus)
lr = LogisticRegression(C=5,
    max_iter=500)
scores = cross_val_score(lr, X, y,
    cv=5)
# mean ~= 0.83
```

Key parameters:

- `sublinear_tf` — replaces f with $1 + \log f$; compresses outlier counts
- `min_df=5` — drops hapax legomena (noise)
- `ngram_range=(1,2)` — bigrams capture “not good”, “highly recommend”
- `C=5` — less regularisation works better on high-dim TF-IDF

Result on NLTK Movie Reviews

≈83% accuracy (5-fold CV)

This is our **target to beat** with dense embeddings.

Why Sparse Vectors Fail at Semantics

Orthogonality problem

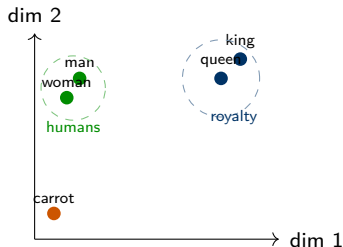
In a 5-word toy vocabulary:

	king	queen	man	woman	carrot
king	1	0	0	0	0
queen	0	1	0	0	0
carrot	0	0	0	0	1

$$\cos(\vec{king}, \vec{queen}) = 0$$

$$\cos(\vec{king}, \vec{carrot}) = 0 \quad \text{Both} = 0!$$

What we want



The Key Insight

Dense embeddings place semantically related words **geometrically close**. Distance and direction both carry meaning.

Firth's Principle (1957)

"You shall know a word by the company it keeps."
— J. R. Firth

Words that appear in similar **contexts**
have similar **meanings**.

- "I ate the _____ for breakfast"
- "She ordered a _____ at the café"
- "The _____ was delicious"

⇒ "croissant", "bagel", "sandwich" are interchangeable here

⇒ They should have **similar vectors**

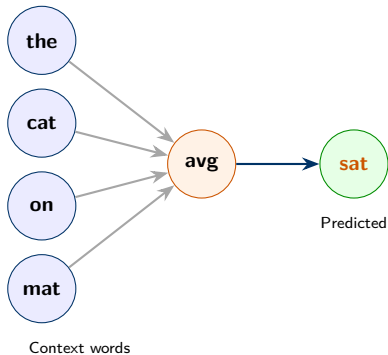
From Hypothesis to Algorithm

- 1 Slide a **context window** of size k over the corpus
- 2 For each (centre, context) pair, **train** a network to predict one from the other
- 3 The learned weights **are** the embeddings

No labels needed — the corpus supervision signal is the text itself.
This is **self-supervised learning**.

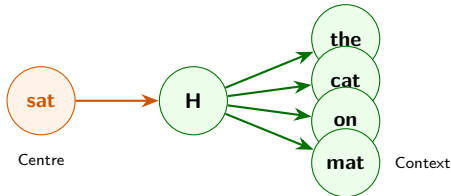
Sentence: the cat [sat] on mat — window $k = 2$, centre word = sat

CBOW — predict centre from context



- Averages context embeddings → hidden

Skip-gram — predict context from centre

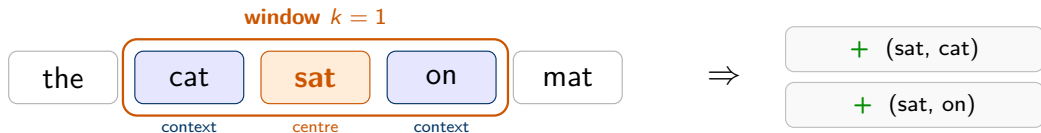


- One centre → 4 training pairs
- More signal; better for **rare** words

Word2Vec — The Sliding Context Window

How Training Pairs Are Generated

A window of half-width k slides over every word in the corpus, generating one (centre, context) pair per context position.



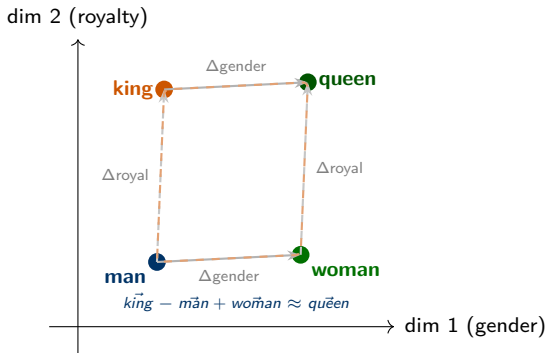
All pairs from “the cat sat on mat”
($k = 1$):

Centre	Context words	Pairs generated
the	cat	(the, cat)
cat	the, sat	(cat, the), (cat, sat)
sat	cat, on	(sat, cat), (sat, on)
on	sat, mat	(on, sat), (on, mat)

Window size effect:

- Small $k = 2$: captures **syntactic** neighbours (subject–verb, noun–adjective)
- Large $k = 10$: captures **topical** co-occurrence (“doctor” near “hospital”)
- Gensim default: window=5
- Dynamic windows: randomly sample

Word2Vec — Embedding Space Geometry



Why does the parallelogram form?

- “king” and “man” share most context words (appeared, said, is. . .)
- “king” *additionally* appears near *throne*, *crown*, *reign*
- Difference $\vec{k\bar{i}ng} - \vec{m\bar{a}n}$ isolates **royalty dimensions**
- Same offset applies:
 $\vec{q\bar{u}e\bar{e}n} - \vec{w\bar{o}m\bar{a}n} \approx \vec{k\bar{i}ng} - \vec{m\bar{a}n}$

Other analogy directions

Capital-country:	Paris – France
Currency:	dollar – USA
Plural:	cats – cat
Past tense:	walked – walk

Benchmark: Mikolov’s analogy test set has 19,544 analogies.

Skip-gram objective: maximise the log-probability of context words:

$$\mathcal{L} = \sum_{t=1}^T \sum_{\substack{-k \leq j \leq k \\ j \neq 0}} \log P(w_{t+j} \mid w_t)$$

where $P(w_o \mid w_l) = \frac{\exp(\mathbf{v}_{w_o}^\top \mathbf{v}_{w_l})}{\sum_{w=1}^W \exp(\mathbf{v}_w^\top \mathbf{v}_{w_l})}$

Negative Sampling [8]

Computing the softmax over $|V| \approx 50\text{K}$ words is expensive. **Negative sampling** replaces it with a binary classifier:

- 1 positive (real) context word
- $k = 5\text{--}20$ random “noise” words

Reduces training cost from $O(|V|)$ to $O(k)$ per sample.

What gets learned:

- **Embedding matrix** $W \in \mathbb{R}^{|V| \times d}$: row i is the vector for word i
- The matrix W is the weight matrix of the hidden layer
- Prediction head is discarded after training
- The geometry of W encodes semantic relationships

Skip-gram: Step-by-Step Numerical Example

Setup

Corpus: "cat sat on mat" Vocabulary: {cat= 0, sat= 1, on= 2, mat= 3}, $|V| = 4$, $d = 2$, window $k = 1$
 Centre word: **sat** $\Rightarrow$ training pairs: (sat, cat), (sat, on)

Step 1 — One-hot input

$$\mathbf{x}_{\text{sat}} = [0, \mathbf{1}, 0, 0]^T$$

Step 2 — Embedding lookup (matrix W)

$W \in \mathbb{R}^{4 \times 2}$ (random init, e.g.):

$$W = \begin{pmatrix} 0.1 & 0.2 \\ 0.3 & -0.1 \\ -0.2 & 0.4 \\ 0.5 & 0.0 \end{pmatrix}$$

$$\mathbf{h} = W^T \mathbf{x}_{\text{sat}} = \text{row}_1(W) = [0.3, -0.1]$$

Step 3 — Output scores (matrix W')

$W' \in \mathbb{R}^{4 \times 2}$ (separate output embeddings):

$$\mathbf{u}_j = W'[j, :] \cdot \mathbf{h}$$

E.g. $W'[\text{cat}] = [0.4, 0.2]$:

$$u_{\text{cat}} = 0.4(0.3) + 0.2(-0.1) = 0.12 - 0.02 = 0.10$$

Step 4 — Softmax & loss

$$P(\text{cat} \mid \text{sat}) = \frac{e^{0.10}}{\sum_j e^{u_j}}$$

$$\text{Loss: } \mathcal{L} = -\log P(\text{cat} \mid \text{sat}) - \log P(\text{on} \mid \text{sat})$$

SGNS — Negative Sampling: Step-by-Step

Why SGNS?

Full softmax costs $O(|V|)$ per sample. SGNS turns each training step into $k+1$ **binary logistic regressions**: 1 positive + k negatives.

Training pairs from "sat"

Centre	Context	Label	Source
sat	cat	1	real window
sat	on	1	real window
sat	mat	0	noise sample
sat	cat	0	noise sample

Noise words drawn $\propto p_{\text{unigram}}^{3/4}$
(sublinear smoothing helps rare words)

Loss for one positive pair (sat, cat)

Let $\sigma(x) = \frac{1}{1+e^{-x}}$ (sigmoid).

Score: $s = \mathbf{v}_{\text{sat}} \cdot \mathbf{v}'_{\text{cat}}$
 $= [0.3, -0.1] \cdot [0.4, 0.2] = 0.10$

$\sigma(0.10) \approx 0.525 \approx 53\%$ "real"

SGNS loss:

$$\mathcal{L} = -\log \sigma(s^+) - \sum_{i=1}^k \log \sigma(-s_i^-)$$

Gradient updates both $\mathbf{v}_{\text{sat}}$ and $\mathbf{v}'_{\text{cat}}$.

SGNS — Noise Distribution & Sampling Visualised

Why not sample uniformly?

In a typical corpus, “the” appears millions of times; “cinematographic” once. Sampling uniformly means negatives are almost always stop-words — useless signal.

$p^{3/4}$ smoothing:

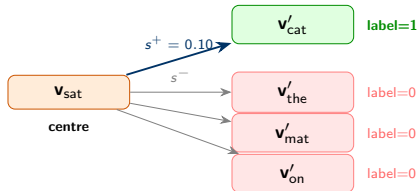
$$P_n(w) \propto p(w)^{3/4}$$

Word	$p(w)$	$p^{3/4}$	Relative boost
the	0.100	0.1778	$\times 1.78$
cat	0.010	0.0316	$\times 3.16$
sat	0.005	0.0188	$\times 3.75$
rare_word	0.001	0.0056	$\times 5.62$

Rare

words get **relatively more** sampling weight — balances learning signal.

Training step visualised (k=3):



Loss = $-\log \sigma(s^+) - \sum_{i=1}^3 \log \sigma(-s_i^-)$
Each step updates **only** $k+1 = 4$ rows of W (not all $|V|$).

GloVe: Step-by-Step Numerical Example

Setup

Tiny corpus: 3 sentences. Co-occurrence matrix X (window = 1): $X_{\text{cat,sat}} = 2$, $X_{\text{sat,on}} = 3$, $X_{\text{on,mat}} = 1$

Step 1 — Co-occurrence entry

Take pair (cat, sat): $X_{ij} = 2$

$\log X_{ij} = \log 2 \approx 0.693$

Step 2 — Weighting function f

$x_{\max} = 100$, $\alpha = 0.75$:

$f(2) = (2/100)^{0.75} \approx 0.027$

Step 3 — Compute residual

Current embeddings (random init):

$\mathbf{w}_{\text{cat}} = [0.5, 0.1]$, $\tilde{\mathbf{w}}_{\text{sat}} = [0.3, 0.4]$

$b_{\text{cat}} = 0.1$, $\tilde{b}_{\text{sat}} = 0.2$

Dot product: $0.5 \cdot 0.3 + 0.1 \cdot 0.4 = 0.19$

Predicted: $0.19 + 0.1 + 0.2 = 0.49$

Residual: $0.49 - 0.693 = -0.203$

Step 4 — Weighted squared loss

Contribution of this pair to J :

$$f(2) \cdot (-0.203)^2 \approx 0.027 \times 0.041 \approx 0.001$$

Step 5 — Gradient update

$\delta = 2 f(X_{ij}) \cdot \text{residual}$

$\Delta \mathbf{w}_{\text{cat}} = -\eta \delta \tilde{\mathbf{w}}_{\text{sat}}$

After many steps over all (i, j) :

$\mathbf{w}_i \cdot \tilde{\mathbf{w}}_j \approx \log X_{ij} - b_i - \tilde{b}_j$

Final embedding

$\mathbf{e}_{\text{cat}} = \mathbf{w}_{\text{cat}} + \tilde{\mathbf{w}}_{\text{cat}}$ (sum of word-as-centre and word-as-context vectors)

GloVe — Co-occurrence Matrix & Weighting Function

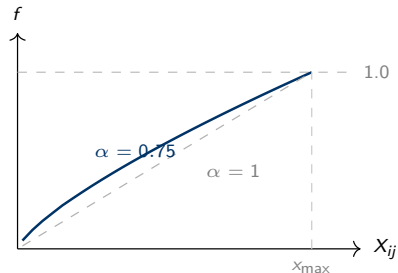
Co-occurrence matrix X (window=1)

Corpus: “the cat sat on mat. the cat ate the fish.”

	the	cat	sat	on	mat
the	0	5	0	0	0
cat	5	0	2	0	0
sat	0	2	0	1	0
on	0	0	1	0	1
mat	0	0	0	1	0

Weighting function $f(X_{ij})$:

$$f(x) = \min\left(1, \left(\frac{x}{x_{\max}}\right)^{\alpha}\right)$$



$\alpha = 0.75$ (default): gives rare pairs **relatively more** weight vs $\alpha = 1$ (pure ratio). Pairs above $x_{\max} = 100$ capped at 1.


```
from gensim.models import Word2Vec
from gensim.models.callbacks import \
    CallbackAny2Vec

class EpochLogger(CallbackAny2Vec):
    def on_epoch_end(self, model):
        loss =
            model.get_latest_training_loss
        print(f"Epoch {self.epoch}:
            loss={loss:.0f}")

sentences =
    [list(movie_reviews.words(f))
      for f in
        movie_reviews.fileids()]

w2v = Word2Vec(
    sentences,
    vector_size=100, window=5,
    min_count=5, sg=1, # Skip-gram
    workers=4, epochs=15,
    compute_loss=True,
    callbacks=[EpochLogger()])
```

Key hyperparameters:

Param	Meaning
vector_size	Embedding dim (50–300)
window	Context half-width
min_count	Drop rare words
sg=1	Skip-gram (0=CBOW)
negative	# noise samples
epochs	Training passes

Rule of thumb:

Train on >1M tokens for useful embeddings. For small corpora, use pre-trained vectors.

Word Similarity & Analogy Arithmetic

Cosine Similarity

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

Pair	Cosine
good — great	≈ 0.84
film — movie	≈ 0.79
good — bad	≈ 0.72
film — president	≈ 0.11

Analogy Arithmetic [8]

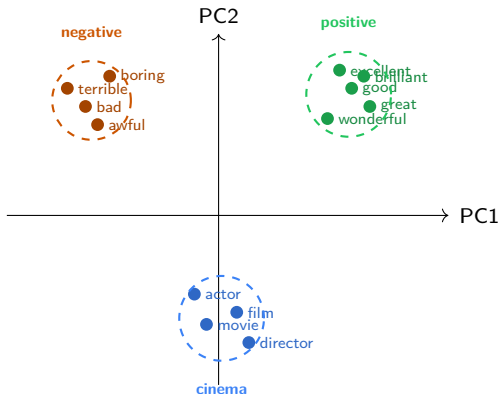
Word vectors support linear offsets:

$$\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$$

$$\vec{Paris} - \vec{France} + \vec{Germany} \approx \vec{Berlin}$$

```
w2v.wv.most_similar(  
    positive=['king', 'woman'],  
    negative=['man'],  
    topn=3)
```

These geometric relationships emerge **automatically** from corpus statistics — they are not hand-coded.



Reading PCA Plots

PCA rotates the 100-dim space so PC1/PC2 capture most variance. Within-cluster distances are meaningful; between-cluster distances may not preserve 100-dim geometry. Always verify with `model.wv.similarity()`.

GloVe — Global Vectors [9]

Core Difference from Word2Vec

Word2Vec trains on *local* context windows only. GloVe uses **global co-occurrence statistics** of the entire corpus.

Step 1 — Build co-occurrence matrix X

- X_{ij} = times word j appears in context of word i
- Computed *once* over the full corpus
- Size $|V| \times |V|$ — stored sparse

Step 2 — Learn embeddings

Minimise the weighted objective:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) (\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

where $f(x) = \min(1, (x/x_{\max})^\alpha)$ caps frequent pairs.

Intuition:

$\mathbf{w}_i \cdot \tilde{\mathbf{w}}_j$ should approximate $\log X_{ij}$ — the log of how often they co-occur.

Why two vectors per word?

$\mathbf{w}$ = word-as-centre; $\tilde{\mathbf{w}}$ = word-as-context.

Final embedding = $\mathbf{w} + \tilde{\mathbf{w}}$.

	W2V	GloVe
Stats used	Local	Global
Training	Online	Batch
Speed	Faster	Slower
Analogies	Good	Better

GloVe — Pre-trained Vectors via Gensim

```
import gensim.downloader as api

# glove-wiki-gigaword-50 (~66 MB)
# glove-wiki-gigaword-100 (~128 MB)
# glove-wiki-gigaword-300 (~376 MB)
glove =
    api.load('glove-wiki-gigaword-50')

print(glove.similarity('king',
    'queen'))
# 0.6510

result = glove.most_similar(
    positive=['king', 'woman'],
    negative=['man'], topn=3)
# queen (0.8934), princess
# (0.7650)...

# Pre-built analogy test set
glove.evaluate_word_analogies(
    'questions-words.txt')
```

Available pre-trained GloVe models:

Model	Corpus	Size
-50	Wiki+Gigaword	66 MB
-100	Wiki+Gigaword	128 MB
-300	Wiki+Gigaword	376 MB
twitter-25	Twitter	48 MB
twitter-200	Twitter	758 MB

When to use GloVe

Prefer GloVe over training Word2Vec from scratch when your corpus has <10M tokens. The pre-trained 6B-token Wikipedia model will produce far better embeddings than training on a small dataset.

FastText — Subword Embeddings [1]

The OOV Problem

Word2Vec and GloVe treat each word as atomic. **Out-of-vocabulary** (OOV) words get no vector — a `KeyError` at inference time. FastText solves this with **character n-gram decomposition**.

Subword decomposition (min=3, max=6):

Word	Subword pieces
playing	<pl, pla, lay, ayi, yin, ing, ng> + <playing>
runs	<ru, run, uns, ns>
NLPify	pieces from unknown word

Word vector = **sum** of subword vectors: $\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$

Benefits:

- 1 **OOV words** get a vector from subword pieces — never zero
- 2 **Morphological variants** share subwords: “walk”, “walks”, “walking”, “walked”
- 3 **Misspellings**: “recieve” shares pieces with “receive”
- 4 **Morphologically rich languages** (Turkish, Finnish, Arabic) benefit most

When to choose FastText

User-generated content (social media, reviews), non-English text, or any domain with many rare/novel terms

FastText Training & OOV Demo

```

from gensim.models import FastText

ft = FastText(
    sentences,
    vector_size=100,
    window=5,
    min_count=5,
    sg=1,                      # Skip-gram
    word_ngrams=1,            # enable
                               subword
    workers=4,
    epochs=10,
)

# OOV word -- no KeyError!
vec = ft.wv['cinematographic']
print(f"OOV vector norm:
      {np.linalg.norm(vec):.4f}")

# Nearest in-vocabulary neighbours
ft.wv.most_similar('cinematographic',
                   topn=5)

```

OOV comparison:

Word	In W2V?	In FastText?
overacting	X	✓
cinematographic	X	✓
storyline	X	✓
pre-trained	X	✓
film	✓	✓

Vocabulary difference:

Word2Vec on movie reviews: $\approx 11\text{K}$ words

FastText: $\approx 31\text{K}$ entries (incl. subword pieces)

Larger vocabulary, but each entry is a subword — not a full word — so memory stays manageable.

From Word Vectors to Document Vectors

Word2Vec/GloVe/FastText produce *word*-level vectors. For classification we need *document*-level vectors.

Method 1 — Mean Pooling (simplest)

$$\mathbf{v}_{\text{doc}} = \frac{1}{|d|} \sum_{w \in d} \mathbf{e}_w$$

- Fast, no extra training
- Ignores word order
- Important words diluted by filler
- Accuracy: $\approx 77\%$ on movie reviews

Key Limitation of Averaging

“not good” averages to a vector *between* “not” and “good”, confusing the sentiment classifier. Order-aware models (LSTMs, Transformers) fix this.

Method 2 — Doc2Vec [6]

$$\mathbf{v}_{\text{doc}} + \mathbf{e}_{w_i} \rightarrow P(w_{i+1})$$

- Trains a *paragraph* vector alongside word vectors
- Captures word order and document context
- More accurate but slower to train

Method 3 — Sentence Transformers [10]

BERT [CLS] or mean-pooled BERT outputs — state of the art.

Mean Pooling — Worked Numerical Example

Sentence: “cat sat mat” (vocabulary of 5: *the, cat, sat, on, mat*; $d = 3$)

Step 1 — Look up each word vector

Word	e_1	e_2	e_3
<i>cat</i>	0.6	0.1	-0.3
<i>sat</i>	0.2	0.8	0.5
<i>mat</i>	0.5	0.2	-0.1

Step 2 — Sum the vectors

$$s = [0.6+0.2+0.5, 0.1+0.8+0.2, -0.3+0.5-0.1]$$

$$= [1.3, 1.1, 0.1]$$

Step 3 — Divide by token count ($n = 3$)

$$\mathbf{v}_{\text{doc}} = \frac{1}{3}[1.3, 1.1, 0.1] = [0.43, 0.37, 0.03]$$

What this vector “knows”:

- Dimension 1 (e_1) is high $\rightarrow$ all three words have a cat-like / concrete-object component in GloVe space.
- Dimension 2 (e_2) is moderate $\rightarrow$ shared verb/action context.
- Dimension 3 (e_3) near zero $\rightarrow$ the negative cat/mat cancel the positive sat.

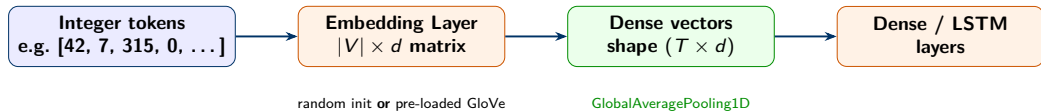
The negation problem

If we add “not” = $[-0.6, -0.1, 0.3]$, dimension 1 drops: $[0.43-0.15, \dots] = [0.28, \dots]$. The word “not” partially *cancels* meaning — but only partially. A classifier cannot reliably detect negation from the average alone.

The Keras Embedding Layer

What It Is

An Embedding layer is a **trainable lookup table**: given integer token IDs, it returns dense vectors. It is the bridge from discrete tokens to continuous neural-network space.



Two initialisation strategies:

- **Random init + trainable=True**: learns from scratch on task labels
- **Pre-loaded GloVe + trainable=False**: freezes the embedding; faster, better for small datasets

Padding:

Reviews have variable length. `pad_sequences(maxlen=200)` truncates long reviews and pads short ones with zeros. A **Masking** layer can tell downstream layers to ignore padding tokens.

```
from keras.preprocessing.text import Tokenizer
from keras.preprocessing.sequence import
    pad_sequences
from keras.models import Sequential
from keras.layers import (Embedding,
    GlobalAveragePooling1D, Dense,
    Dropout, BatchNormalization)
MAX_VOCAB, MAX_LEN = 10_000, 200
# Build GloVe weight matrix
emb_matrix = np.zeros((MAX_VOCAB+1, 50))
for word, idx in tok.word_index.items():
    if idx < MAX_VOCAB+1 and word in glove:
        emb_matrix[idx] = glove[word]
model = Sequential([
    Embedding(MAX_VOCAB+1, 50,
        weights=[emb_matrix], trainable=False),
    GlobalAveragePooling1D(),
    Dense(64, activation='relu'),
    BatchNormalization(),
    Dropout(0.4),
    Dense(1, activation='sigmoid'),
])
model.compile('adam', 'binary_crossentropy',
    metrics=['accuracy'])
```

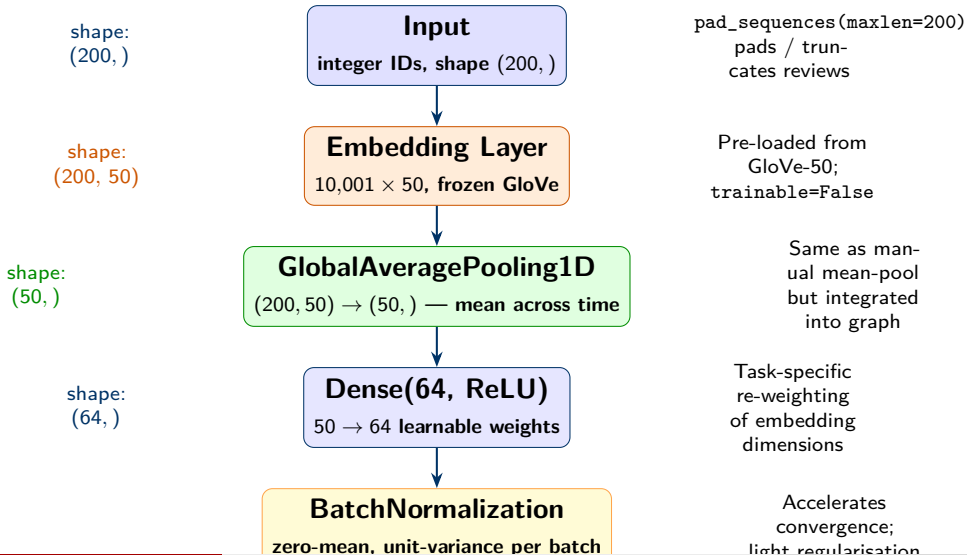
Architecture notes:

- GlobalAveragePooling1D — mean-pools across the sequence dimension; equivalent to the manual averaging in Section 8
- BatchNormalization — normalises layer inputs; accelerates training and acts as light regularisation
- Dropout(0.4) — randomly zeros 40% of neurons during training; prevents co-adaptation
- trainable=False — GloVe weights are frozen; only Dense layers are updated

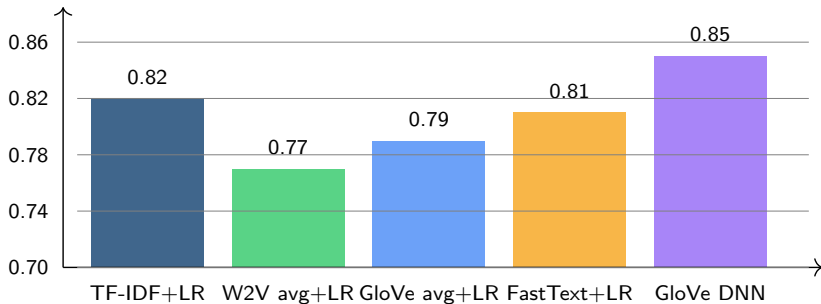
Result

Best val. accuracy $\approx 83\text{--}87\%$ depending on training epochs and GloVe dimension.

GloVe DNN — Architecture Visualisation



Method Comparison — Movie Review Sentiment



Key Observations

- TF-IDF is surprisingly competitive — keyword signal is preserved
- Mean pooling dilutes discriminating words; DNN with GloVe recovers them
- 83% → 98%+ requires contextual models (BERT) — Session 4

Decision Guide — Which Method to Use?

Scenario	Recommended method	Why
Small dataset (<10K docs)	TF-IDF + LR / SVM	Low variance; interpretable
Keyword / search retrieval	TF-IDF	Each dim is a term
Semantic similarity	GloVe / W2V pre-trained	Nearest-neighbour in embedding space
Rare words / misspellings	FastText	Subword handles OOV
Morphologically rich lang.	FastText	Character pieces share across inflections
Medium labelled corpus	GloVe DNN (frozen)	Pre-trained signal + task fine-tuning
Large labelled corpus	Embedding trainable=True	Learn task-specific geometry
State-of-the-art	BERT / RoBERTa	Session 4

Common Mistakes

- Fitting vectorizer on full data (leaks test vocab)
- Training W2V on <1M tokens (use pre-trained)
- Averaging without normalising (scale matters)
- Not tuning max_features (memory blown)

Evaluation Checklist

- Always use stratified K-fold CV
- Report mean ± std, not single run
- Compare macro-F1, not just accuracy
- Check per-class performance (imbalanced data)

Session Summary

Concepts covered:

- 1 BoW / TF-IDF: sparse baselines, interpretable
- 2 Distributional hypothesis — context defines meaning
- 3 Word2Vec: CBOW / Skip-gram, negative sampling
- 4 GloVe: global co-occurrence objective
- 5 FastText: subword n-grams, OOV-robust
- 6 Document vectors via mean-pooling
- 7 Keras Embedding layer (frozen or trainable)
- 8 Method comparison on movie reviews

Key formulae:

$$\text{Skip-gram: } \max_t \sum_j \log P(w_{t+j} | w_t)$$

$$\text{GloVe: } \min_{i,j} \sum f(X_{ij})(w_i^T \tilde{w}_j - \log X_{ij})^2$$

$$\text{FastText: } \mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$$

$$\text{Mean-pool: } \mathbf{v}_{doc} = \frac{1}{|d|} \sum_{w \in d} \mathbf{e}_w$$

Next Session

Sequence-to-Sequence models — Encoder-Decoder, Attention mechanism, Neural Machine Translation (NMT).

Further Reading & Resources

Foundational Papers

- Mikolov et al. (2013) — Word2Vec [8]
- Pennington et al. (2014) — GloVe [9]
- Bojanowski et al. (2017) — FastText [1]
- Le & Mikolov (2014) — Doc2Vec [6]
- Levy & Goldberg (2014) — W2V as matrix factorisation [7]
- Harris (1954) — Distributional structure [4]

Textbooks

- Jurafsky & Martin — SLP3 ch.6 (Vector Semantics) [5]
- Goldberg — Neural Network Methods for NLP [3]

Online Resources

- Stanford GloVe project page
- Mikolov's original Word2Vec code
- FastText official site (Meta AI)
- Gensim documentation
- TF Embedding Projector (visual)
- Jay Alamar — Illustrated Word2Vec

Notebooks / Practicals

- HuggingFace NLP Course — Chapter 2 (Tokenizers)
- TensorFlow Word2Vec tutorial
- Gensim Word2Vec tutorial (official docs)

References I



P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov.

Enriching word vectors with subword information.

Transactions of the Association for Computational Linguistics, 5:135–146, 2017.



J. R. Firth.

A synopsis of linguistic theory 1930–1955.

Studies in Linguistic Analysis, pages 1–32, 1957.



Y. Goldberg.

Neural Network Methods for Natural Language Processing.

Morgan & Claypool, 2017.



Z. S. Harris.

Distributional structure.

Word, 10(2–3):146–162, 1954.



D. Jurafsky and J. H. Martin.

Speech and Language Processing.

3rd (draft) edition, 2024.

Available at <https://web.stanford.edu/~jurafsky/slp3/>.



Q. V. Le and T. Mikolov.

Distributed representations of sentences and documents.

In *Proceedings of ICML*, pages 1188–1196, 2014.



O. Levy and Y. Goldberg.

Neural word embedding as implicit matrix factorization.

In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, 2014.

References II



[T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean.](#)

Distributed representations of words and phrases and their compositionality.

In Advances in Neural Information Processing Systems (NeurIPS), volume 26, 2013.



[J. Pennington, R. Socher, and C. D. Manning.](#)

GloVe: Global vectors for word representation.

In Proceedings of EMNLP, pages 1532–1543, 2014.



[N. Reimers and I. Gurevych.](#)

Sentence-BERT: Sentence embeddings using Siamese BERT-networks.

In Proceedings of EMNLP-IJCNLP, pages 3982–3992, 2019.